

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Определение характеристик графов»

Выполнили:
студенты группы 24ВВВЗ
Назаров Н.Д.
Майер В.С.
Савин А.В.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., Деев М.В.

Пенза 2025

Цель работы – Освоение методов программной обработки графовых структур путем реализации алгоритмов преобразования матричных представлений графов и анализа их структурных характеристик.

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Определите размер графа G , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Код программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <conio.h>
#include <locale.h>

int countEdges(int** matrix, int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            if (matrix[i][j] == 1) count++;
        }
    }
    return count;
}

void analyzeVertices(int** matrix, int size) {
    printf("\nАнализ вершин:\n");

    for (int i = 0; i < size; i++) {
        int degree = 0;

        for (int j = 0; j < size; j++) {
            if (matrix[i][j] == 1) degree++;
        }

        if (degree == 0) {
            printf("Вершина %d: Изолированная (степень: 0)\n", i);
        }
        else if (degree == 1) {
            printf("Вершина %d: Концевая (степень: 1)\n", i);
        }
        else if (degree == size - 1) {
            printf("Вершина %d: Доминирующая (степень: %d)\n", i, degree);
        }
        else {
            printf("Вершина %d: Обычная (степень: %d)\n", i, degree);
        }
    }
}

void makeSymmetric(int** matrix, int size) {
    for (int i = 0; i < size; i++) {
```

```

        for (int j = i + 1; j < size; j++) {

            if (matrix[i][j] == 1) {
                matrix[j][i] = 1;
            }
            else if (matrix[j][i] == 1) {
                matrix[i][j] = 1;
            }
        }
    }
}

void printMatrix(int** matrix, int size) {
    printf("\nМатрица смежности неориентированного графа G:\n ");
    for (int j = 0; j < size; j++) printf("%2d ", j);
    printf("\n");

    for (int i = 0; i < size; i++) {
        printf("%d: ", i);
        for (int j = 0; j < size; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    setlocale(LC_ALL, "Russian");

    int size;
    printf("Введите размер матрицы: ");
    if (scanf("%d", &size) != 1 || size <= 0) {
        printf("Ошибка: неверный размер матрицы\n");
        return 1;
    }

    int** matrix = (int**)malloc(size * sizeof(int*));
    if (matrix == NULL) {
        printf("Ошибка выделения памяти\n");
        return 1;
    }

    for (int i = 0; i < size; i++) {
        matrix[i] = (int*)calloc(size, sizeof(int));
        if (matrix[i] == NULL) {
            printf("Ошибка выделения памяти\n");
            return 1;
        }
    }

    srand(time(NULL));

    printf("\nГенерация неориентированного графа...\n");
    for (int i = 0; i < size; i++) {
        for (int j = i + 1; j < size; j++) {
            matrix[i][j] = rand() % 2;
        }
    }

    makeSymmetric(matrix, size);

    printMatrix(matrix, size);

    int edgeCount = countEdges(matrix, size);
    printf("\nРазмер графа G (количество ребер): %d\n", edgeCount);
}

```

```

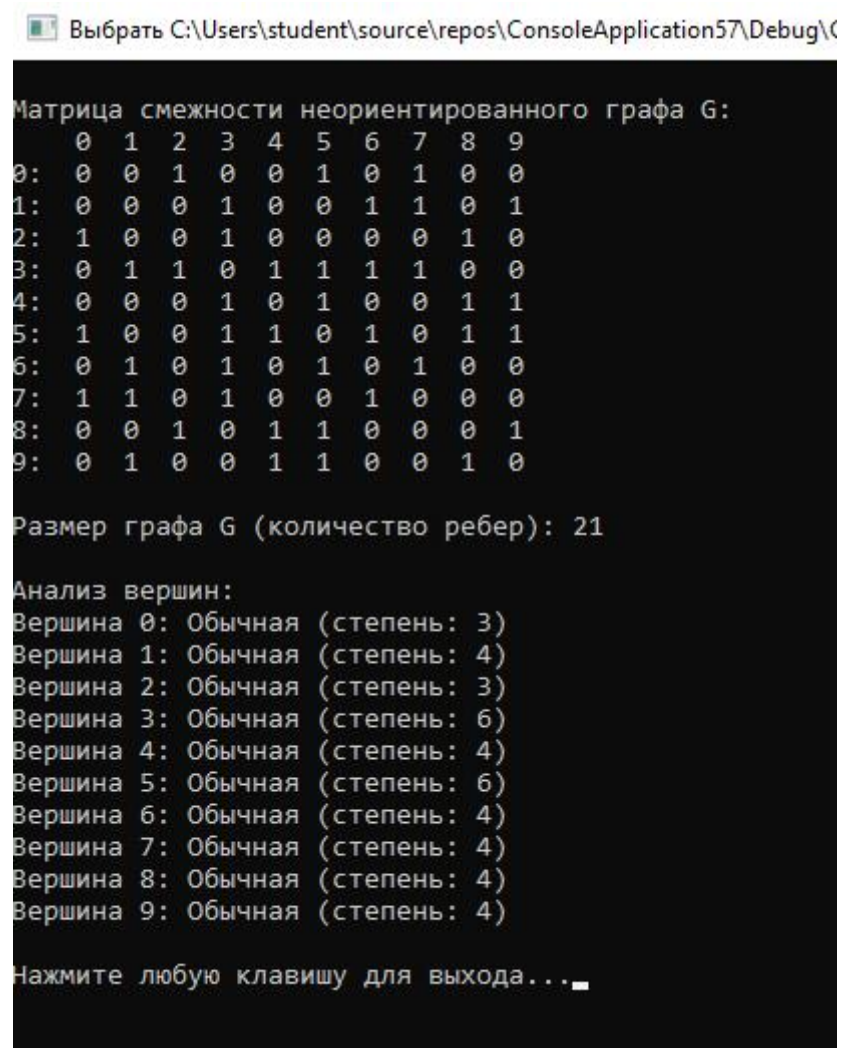
analyzeVertices(matrix, size);

for (int i = 0; i < size; i++) {
    free(matrix[i]);
}
free(matrix);

printf("\nНажмите любую клавишу для выхода...");
_getch();
return 0;
}

```

Результаты работы программы



Выбрать C:\Users\student\source\repos\ConsoleApplication57\Debug\...

```

Матрица смежности неориентированного графа G:
  0  1  2  3  4  5  6  7  8  9
0:  0  0  1  0  0  1  0  1  0  0
1:  0  0  0  1  0  0  1  1  0  1
2:  1  0  0  1  0  0  0  0  1  0
3:  0  1  1  0  1  1  1  1  0  0
4:  0  0  0  1  0  1  0  0  1  1
5:  1  0  0  1  1  0  1  0  1  1
6:  0  1  0  1  0  1  0  1  0  0
7:  1  1  0  1  0  0  1  0  0  0
8:  0  0  1  0  1  1  0  0  0  1
9:  0  1  0  0  1  1  0  0  1  0

Размер графа G (количество ребер): 21

Анализ вершин:
Вершина 0: Обычная (степень: 3)
Вершина 1: Обычная (степень: 4)
Вершина 2: Обычная (степень: 3)
Вершина 3: Обычная (степень: 6)
Вершина 4: Обычная (степень: 4)
Вершина 5: Обычная (степень: 6)
Вершина 6: Обычная (степень: 4)
Вершина 7: Обычная (степень: 4)
Вершина 8: Обычная (степень: 4)
Вершина 9: Обычная (степень: 4)

Нажмите любую клавишу для выхода...

```

Рисунок 1 - Результат работы программы

Вывод: В результате выполнения работы были успешно реализованы алгоритмы преобразования матрицы смежности в матрицу инцидентности и разработаны методы анализа вершин графа.